

The Attention Matrix, Entry by Entry

One GPT-2 head, computed by hand · video 2:55

This lesson answers one question: where does each number in an attention diagram come from? One head of the real GPT-2 is recomputed by hand on *The cat sat on the mat because it was tired*: queries, keys and values; one score as a sum of 64 products, divided by 8; the mask and the softmax; the weighted sum of values; and the 12 heads joined back together. Every step matches the `transformers` library exactly.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Queries, keys and values	▶ 0:08-0:42	2	☐
2. One entry, one row	▶ 0:42-1:13	2	☐
3. The matrix, and why divide by 8	▶ 1:13-1:48	3	☐
4. Values, heads, and scale	▶ 1:48-2:47	3	☐

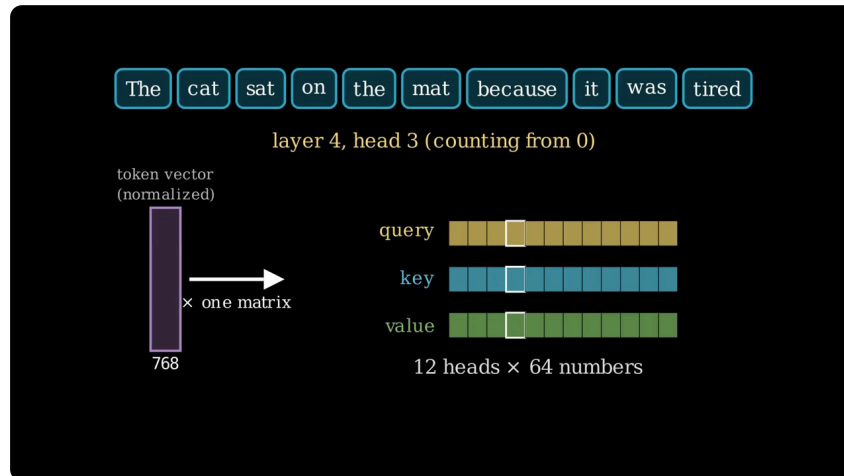
Before you start: How LLMs Work episodes 5-6 (attention) and Deep Dive episode 7 (the causal mask). Code: `code/d08_attention_matrix` (GPT-2 small, about 500 MB).

CONCEPT 1 OF 4

Queries, keys and values

▶ Video 0:08–0:42

The sentence has ten tokens. We follow **layer 4, head 3** of GPT-2 small (counting from zero). Each token's 768-number vector is first normalized (LayerNorm), then multiplied by one matrix that produces its **query**, **key** and **value**, 768 numbers each. Each is split into **12 heads of 64 numbers**; head 3 uses numbers 192–255 of each.



▶ 0:40 One matrix turns each normalized 768-number token vector into a query, a key and a value, each split into 12 heads of 64.

KEY IDEA

A head is just a 64-number slice of the queries, keys and values; all 12 heads are computed by the same matrix multiplication.

Check 1: Queries, keys and values

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

GPT-2's query-key-value matrix maps 768 numbers to 3×768 . How many parameters does it have, including its 2,304 biases?

.....

1.2 CALCULATION

How many numbers per head would a model with 1,024-number vectors and 16 heads have?

.....

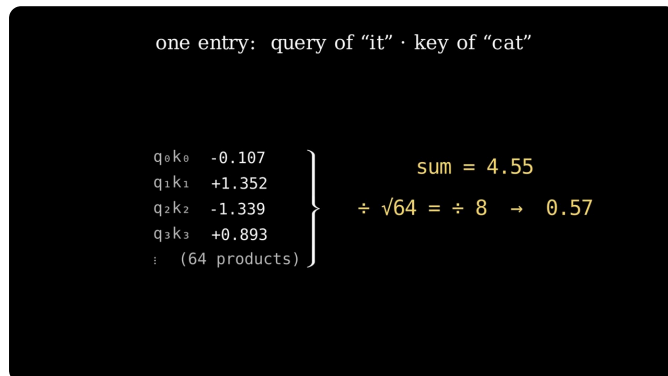
CONCEPT 2 OF 4

One entry, one row

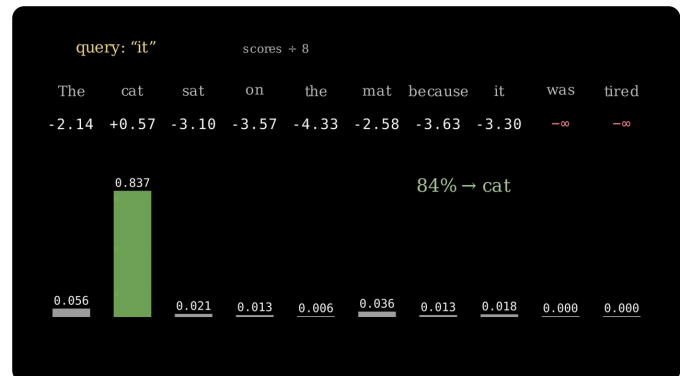
▶ Video 0:42-1:13

One entry: the query of *it* against the key of *cat*. Multiply the 64 pairs of numbers and add them up: **4.55**. Divide by 8, the square root of 64: **0.57**. That is one score.

Do the same for every token up to *it*: the scores range from -4.33 (*the*) to $+0.57$ (*cat*). The future (*was*, *tired*) is masked with $-\infty$, softmax turns the row into weights, and **0.837** goes to *cat*. In this head, *it* points at the cat.



▶ 0:56 Query of "it" times key of "cat": 64 products add up to 4.55; divided by 8, 0.57.



▶ 1:11 The whole row for "it": mask the future, apply softmax, and 84% of the attention goes to "cat".

KEY IDEA

Each entry is a dot product divided by $\sqrt{(\text{head size})}$; each row is a softmax over the tokens a query is allowed to see.

Check 2: One entry, one row

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

If "it" could see only "The" (score -2.14) and "cat" (score 0.57), what weight would "cat" get?

2.2 HANDS-ON CODE

Try it yourself. Using `qh` and `kh` from `attention_matrix.py`, compute the same score with a single matrix product and check it.

```
s = qh[HEAD] @ kh[HEAD].T / 8
print(s[i, j], (qh[HEAD, i] * kh[HEAD, j]).sum() / 8)
```

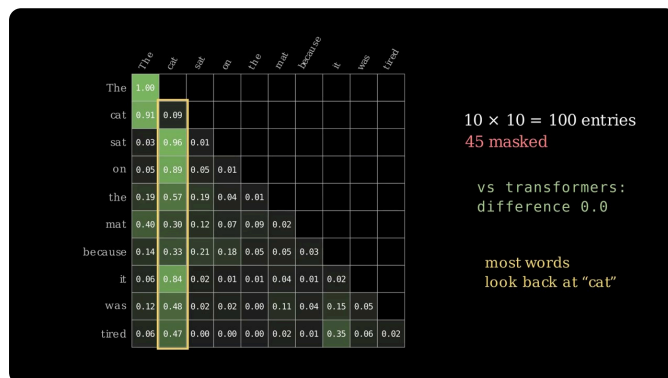
CONCEPT 3 OF 4

The matrix, and why divide by 8

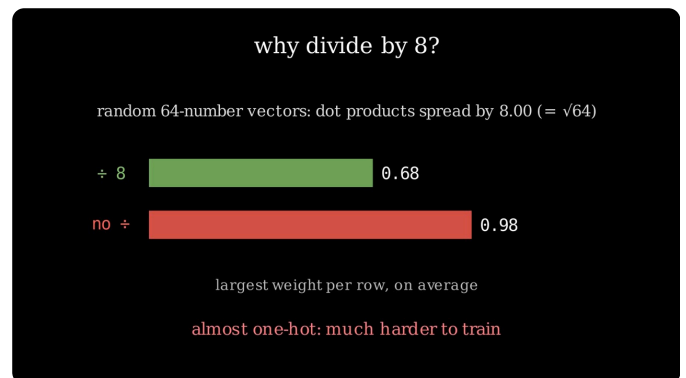
▶ Video 1:13-1:48

Repeat for every query and you get the full matrix: $10 \times 10 = 100$ entries, 45 of them masked. The hand-computed weights match the library's with a largest difference of **0.0**. In this head, most words look back at *cat*, the subject of the sentence.

Why divide by 8? The dot product of two random 64-number vectors spreads out by about 8 (the code measures a standard deviation of 8.00, which is $\sqrt{64}$). Without the division, softmax becomes nearly all-or-nothing: the largest weight in each row averages **0.98** instead of **0.68**. Almost one-hot, so the gradients for every other token nearly vanish, and training is much harder.



▶ 1:27 The full 10×10 matrix: 45 entries masked, most rows pointing back at "cat"; identical to transformers.



▶ 1:47 Without dividing by 8, the largest weight per row averages 0.98 instead of 0.68: almost one-hot.

KEY IDEA

Dividing by $\sqrt{(\text{head size})}$ keeps scores in a range where softmax stays soft, whatever the head size.

Check 3: The matrix, and why divide by 8

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

A model uses heads of 128 numbers. By what does it divide each score?

.....

3.2 TRUE OR FALSE

"Without the division by 8, the row for "it" in this head would still give "cat" about 84%."

True ☐ False ☐

3.3 CALCULATION

In a causal 10×10 attention matrix, how many entries are not masked?

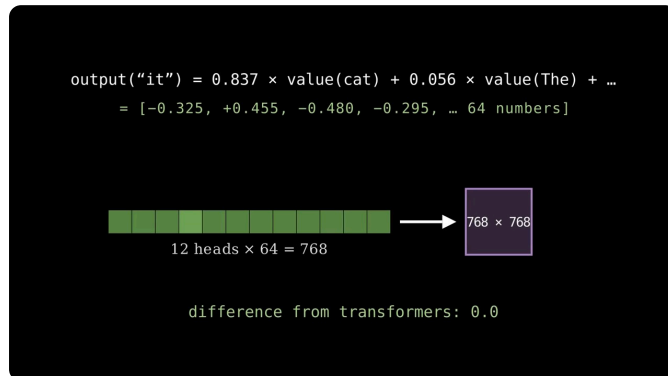
.....

CONCEPT 4 OF 4

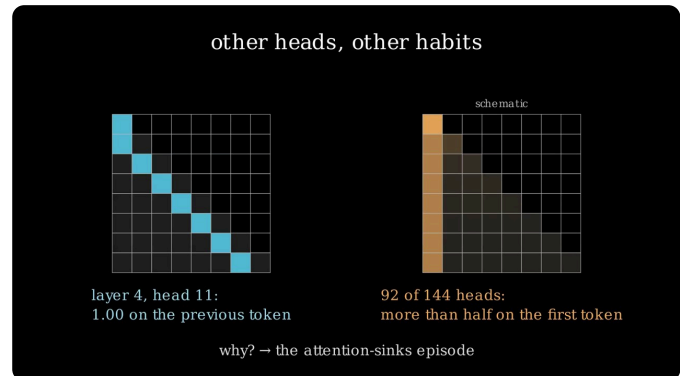
Values, heads, and scale

▶ Video 1:48-2:47

The weights then mix the **values**: the output for *it* is $0.837 \times \text{value}(\text{cat}) + 0.056 \times \text{value}(\text{The}) + \dots$, 64 numbers, mostly cat's. The 12 heads' outputs are joined back into 768 numbers and mixed by one more 768×768 matrix. Again identical to the library: a difference of 0.0.



▶ 2:03 The weights mix the values; 12 heads $\times$ 64 = 768 numbers, mixed by one 768×768 matrix.



▶ 2:21 Other heads: one looks only at the previous token; in 92 of 144, most attention goes to the first token (schematic on the right).

Other heads have other habits. In **layer 4, head 11**, every token puts all of its attention (1.00) on the token just before it. In **92 of the 144** heads, more than half the attention goes to the very first token (the attention-sinks episode explains why).

GPT-2 builds $12 \times 12 = \mathbf{144}$ of these matrices on every forward pass; at 1,024 tokens that is **150,994,944** weights. Keeping that memory in check is the subject of the FlashAttention episode.

KEY IDEA

A head reads the values of the tokens it attends to; the output matrix then combines what all the heads found.

Check 4: Values, heads, and scale

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 PUT IN ORDER

Put one head's steps in order: *softmax · multiply by the values · dot products of queries and keys · divide by $\sqrt{64}$ · mask the future.*

4.2 CALCULATION

How many attention weights would GPT-2 compute in one forward pass at 512 tokens?

4.3 SHORT ANSWER

What does a "previous-token head" make possible for the layers above it?

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Queries, keys and values · rewatch ▶ 0:08-0:42

1.1 1,771,776. $768 \times 2,304 = 1,769,472$ weights, plus 2,304 biases.

1.2 64. $1,024 / 16 = 64$, the same head size as GPT-2.

2. One entry, one row · rewatch ▶ 0:42-1:13

2.1 About 0.938. $e^{0.57} / (e^{0.57} + e^{-2.14}) \approx 0.938$; fewer competitors, more weight.

2.2 Both print 0.569 (shown as 0.57). A matrix product computes every query-key dot product at once; entry (i, j) is exactly the hand computation.

3. The matrix, and why divide by 8 · rewatch ▶ 1:13-1:48

3.1 About 11.31. $\sqrt{128} \approx 11.31$.

3.2 False. Scores 8 times larger make softmax sharper: “cat” gets 1.000 (checked by running it).

3.3 55. $10 + 9 + \dots + 1 = 55$; the other 45 are masked.

4. Values, heads, and scale · rewatch ▶ 1:48-2:47

4.1 dot products → divide by $\sqrt{64}$ → mask the future → softmax → multiply by the values. This is the order in the code (the mask must come before the softmax).

4.2 37,748,736. $144 \times 512 \times 512$; a quarter of the 1,024-token figure, since the matrix grows with the square of the length.

4.3 Each position now carries information about the token just before it, so later heads can match pairs or patterns of tokens, not just single tokens. It is a building block of induction heads (Deep Dive part 10).